



Amberg Infra7D
Trockenloostrasse 21
8105 Regensdorf-Watt
Switzerland

Phone +41 44 870 92 22
Fax +41 44 870 06 18
www.amberg.ch

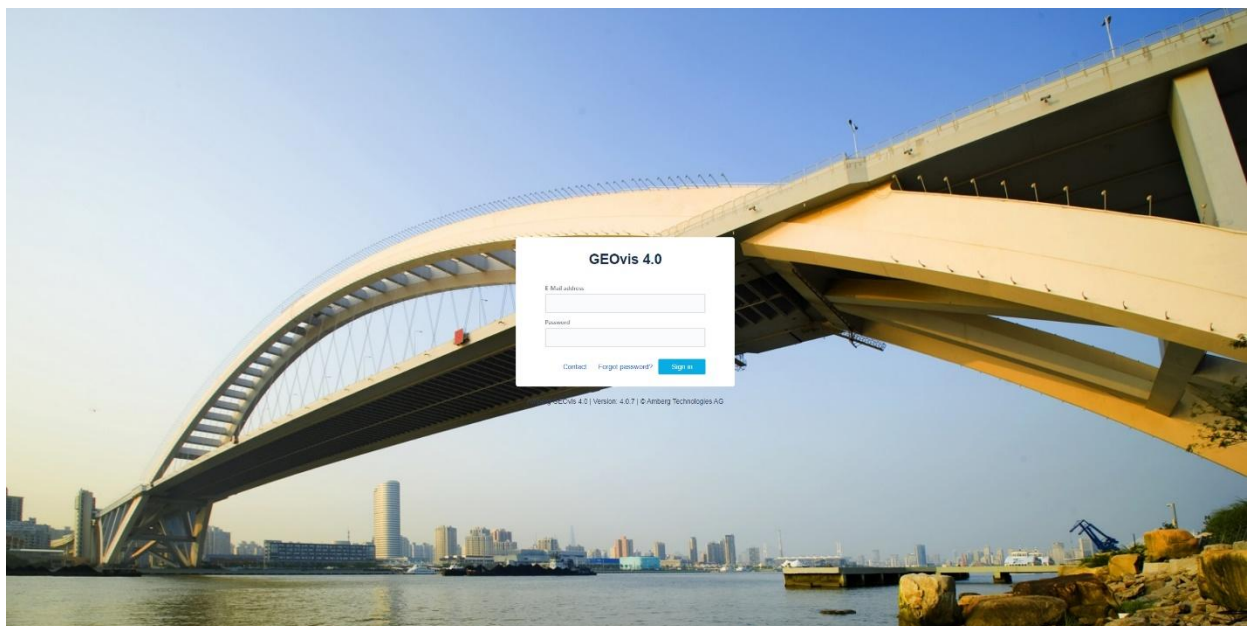
UID No. CHE-292.311.957
VAT No. CHE-116.289.290 MWST

GEOvis REST API

Version 1.5.7

40-G-LFC

Regensdorf, 04.12.2025



Contents**Page**

1. Common information	4
2. GEOvis REST API.....	4
2.1. General response description.....	4
2.2. Login in GEOvis System via REST API	5
2.3. List of available projects request	6
2.4. Project info request	7
2.5. Get sensors names request	8
2.6. Get sensors data request	9
2.7. Get sensors properties request	10
2.8. Sync sensors properties request	12
2.9. Sync sensors data request	13
2.10. Change sensors data state request.....	14
2.11. Delete sensors data request.....	14
2.12. Get views of project request	15
2.13. Get project view's info request	16
2.14. Add sensors to view request	17
2.15. Remove sensors from view request	18
2.16. Get logbook labels request.....	19
2.17. Sync logbook labels request.....	20
2.18. Get logbook request.....	21
2.19. Sync logbook request.....	22
2.20. Get inspection objects of project.....	23
2.21. Get inspection object info	24
3. Used enums descriptions.....	25
3.1. AuthorizationCode.....	25
3.2. ProjectAutomaticState	25
3.3. ProjectManualState	25
3.4. SensorCategory	26
3.5. PhysicalUnit	27
3.6. SensorState	29
3.7. ValuesContent.....	29
3.8. RequestedValueType.....	29
3.9. DistanceUnit.....	30
3.10. SensorProperties	31
3.11. CoordinatesSystems	31
3.12. SyncType	33
3.13. SyncDataOverwriteType.....	33
3.14. ExportDataDensityType	34
3.15. ExportDataPeriod.....	34
3.16. BucketReportValueKind	34
3.17. BucketIntervalType.....	35
3.18. InterpolationAlgorithmMode.....	35
3.19. MongoDataState	35
3.20. SensorMode.....	35
3.21. RequestedValueTime	35
3.22. RequestedValueTimeFormat.....	37
3.23. LogbookType	37
3.24. LogbookConfirmed	37
3.25. LogbookPublic.....	37

04.12.2025

4. Objects structures description.....	38
4.1. ProjectObserveModel	38
4.2. DatabaseModel	38
4.3. UserModel	38
4.4. GeoLocation	39
4.5. ProjectInfoModel	39
4.6. SensorsSelectionFilter	40
4.7. SensorObserveModel	40
4.8. SensorsDataFilter	41
4.9. SensorData	42
4.10. SensorDataModel	42
4.11. ExtendedExportOptions	43
4.12. SensorPropertiesPayload	43
4.13. SensorPropertiesResponse	44
4.14. SyncRequest	46
4.15. DataItem	47
4.16. SensorAttributeEntry	47
4.17. SensorPropertyModel	48
4.18. SensorConfiguration	49
4.19. DistanceModel	50
4.20. AlarmSensorModel	50
4.21. ProjectViewInfo	50
4.22. ProjectViewObserveInfo	51
4.23. DatabaseSensors	51
4.24. SyncStatesPayload	51
4.25. DeleteSensorsDataPayload	52
4.26. BaseSensorsPayload	53
4.27. LabelPayload	54
4.28. LabelResponse	54
4.29. LogbookFilter	54
4.30. LogbookPayload	55
4.31. LogbookResponse	55
4.32. InspectionDatabaseModel	56
4.33. InspectionObjectShortInfo	57
4.34. InspectionObjectInfo	57
Versions changes	58

04.12.2025

1. Common information

GEOvis REST API provides an interface to communicate with GEOvis Server without using GEOvis or GEOvis-Live UI.

Some common rules for GEOvis REST API

- JSON document is used as data transfer format. Both incoming (payload of requests) and outgoing (responses) data has JSON format
- Every request starts with <https://geovis.io/api/rest>. Specific part of each request will be described below.
- Every user stored in GEOvis DB can access GEOvis Server via REST API only if he has permissions to do it. These permissions can be given only by Amberg Admins.
- There are two levels of access permissions:
 - **READ ONLY** (the lowest)
 - **READ WRITE** (the highest).
- In addition to permissions levels each REST API user can be restricted in using databases. A specific number of databases can be set for each user by Amberg Admins. In this case all results which includes data from not allowed databases will not be sent and requests to change something in these databases will be ignored.

2. GEOvis REST API

2.1. General response description

Every request to GEOvis REST API returns the response as JSON document with following (sample) structure:

```
{  
  "Data": SOME_DATA_HERE,  
  "Success": true,  
  "AuthorizationCode": 0,  
  "Messages": [],  
  "JsonPath": ""  
}
```

Content description

- **"Data"** - field with all response's business data. Type: *object*.
- **"Success"** - flag of successful request. Type: *boolean*.
- **"AuthorizationCode"** - code of error. Type: [AuthorizationCode](#)
- **"Messages"** - array of all error messages happened during request execution. In case of successful request is empty. Type: *string[]*.
- **"JsonPath"** - service field. Type: *string*.

04.12.2025

2.2. Login in GEOvis System via REST API

To access all features of GEOvis REST API the user must be logged in GEOvis System. **Login request** is using for authorization users in system.

Request URL

<https://geovis.io/api/rest/login>

Minimum access level required

No level

Request type

POST

Request payload

Sample:

```
{
  "Login": "user@email.com",
  "Password": "user_passw0rd"
}
```

Content description:

- **"Login"** - email of user to be authorized.
- **"Password"** - password of user to be authorized.

Response

Sample:

```
{
  "Data": "long_string_value_token",
  "Success": true,
  "AuthorizationCode": 0,
  "Messages": [],
  "JsonPath": ""
}
```

Content data description:

Bearer Auth token is sending as a business data of Login request. This *token* should be used in Auth header of each following request of logged user. **Time of live of the Auth token is 1 hour.** After this time user has to repeat Login request to get fresh *Auth token*.

04.12.2025

2.3. List of available projects request

To receive all available projects user should send Get Projects request.

Request URL

<https://geovis.io/api/rest/projects>

Minimum access level required

READ ONLY

Request type

GET

Request payload

No payload needed.

Response

Sample:

```
{
  "Data": [
    {
      "Id": 1,
      "Name": "Project 1",
      "Number": "PR_1"
    },
    {
      "Id": 2,
      "Name": "Project 2",
      "Number": "PR_2"
    },
    ...
    {
      "Id": 999,
      "Name": "Project 999",
      "Number": "PR_999"
    }
  ],
  "Success": true,
  "AuthorizationCode": 0,
  "Messages": [],
  "JsonPath": ""
}
```

Content data description:

Array of [ProjectObserveModel](#)

04.12.2025

2.4. Project info request

To receive info about specified project user should send Project Info request.

Request URL

<https://geovis.io/api/rest/project/{projectId:int}>

Minimum access level required

READ ONLY

Request type

GET

Request payload

No payload needed.

Response

Sample:

```
{
  "Data": {
    "Id": 1,
    "Health": [0,2,1],
    "CoordinateSystem": "1",
    "CustomerCompany": "Customer_Company_Name",
    "Location": {
      "Longitude": 8.47448229789734,
      "Latitude": 47.440043819152663,
      "Height": 0.0
    },
    "Leader": {
      "FullName": "Leader, Name",
      "Email": "leader@email.com",
      "Phone": "+123456789"
    },
    "Databases": [
      {
        "Id": "1_0",
        "Name": "DB_0"
      },
      {
        "Id": "1_1",
        "Name": "DB_1"
      },
      ...
      {
        "Id": "1_999",
        "Name": "DB_999"
      }
    ],
    "State": 0
  },
  "Success": true,
  "AuthorizationCode": 0,
  "Messages": [],
  "JsonPath": ""
}
```

04.12.2025

*Content data description:*Instance of [ProjectInfoModel](#)

2.5. Get sensors names request

Receive all sensors match the filter.

Request URL

<https://geovis.io/api/rest/sensors/{projectId:int}>

Minimum access level required

READ ONLY

Request type

POST

Request payload

Instance of [SensorsSelectionFilter](#)

Response

Sample:

```
{
  "Data": [
    {
      "SensorFullId": "DatabaseID.SensorID",
      "Id": "1",
      "CustomerName": "Sensor 1"
    },
    {
      "SensorFullId": "DatabaseID.SensorID",
      "Id": "qwer",
      "CustomerName": "QWER sensor"
    },
    ...
    {
      "SensorFullId": "DatabaseID.SensorID",
      "Id": "N",
      "CustomerName": "Sensor N"
    }
  ],
  "Success": true,
  "AuthorizationCode": 0,
  "Messages": [],
  "JsonPath": ""
}
```

*Content data description:*Array of [SensorObserveModel](#)

04.12.2025

2.6. Get sensors data request

Receive all sensors data match the filter.

Request URL

<https://geovis.io/api/rest/sensorsData/{projectId:int}>

Minimum access level required

READ ONLY

Request type

POST

Request payload

Instance of [SensorsDataFilter](#)

Response

Sample:

```
{
  "Data": [
    {
      "SensorFullId": "DatabaseID.SensorID",
      "Data": [
        {
          "Timeslot": "2021-01-19T00:06:00+03:00",
          "Values": [
            -1.9
          ]
        },
        {
          "Timeslot": "2021-01-19T01:06:00+03:00",
          "Values": [
            -1.9
          ]
        },
        {
          "Timeslot": "2021-01-19T01:36:00+03:00",
          "Values": [
            -2.0
          ]
        }
      ]
    }
  ],
  "Success": true,
  "AuthorizationCode": 0,
  "Messages": [],
  "JsonPath": ""
}
```

Content data description:

Data is array of [SensorDataModel](#)

04.12.2025

2.7. Get sensors properties request

Receive specified properties for each sensor in list

Request URL

<https://geovis.io/api/rest/sensorsProperties/{projectId:int}>

Minimum access level required

READ ONLY

Request type

POST

Request payload

Instance of [SensorPropertiesPayload](#)

Response

Sample:

```
{
  "Data": [
    {
      "SensorFullId": "DatabaseID.SensorID",
      "Properties": {
        "SensorType": 13,
        "CustomerName": "",
        "Unit": 2,
        "PublicFlag": true,
        "Coordinates": {
          "Longitude": 8.2146318885368981,
          "Latitude": 48.84660510513914,
          "Height": 121.223
        },
        "CoordinatesInWGS84": {
          "Longitude": 8.2146318885368981,
          "Latitude": 48.84660510513914,
          "Height": 121.223
        },
        "CoordinatesSystem": 0,
        "Color": "#FAA3F8",
        "UseColorToMapFlag": false,
        "Distance": {
          "Value": 0.0,
          "Unit": 1
        },
        "Param1": "",
        "Param2": "",
        "Param3": "",
        "Param4": "",
        "Param5": "",
        "LastValue": "06.05.2021 13:50:00: 2671812,1056 ",
        "LastAxisValue": "",
        "Offset": null,
        "AxisOffset": null,
        "Reference": {
          "date": "2021-04-23T11:23:00+03:00",
          "values": [
            2671812.1077
          ]
        }
      }
    }
  ]
}
```

04.12.2025

```

    ]
  },
  "AxisReference": null,
  "LastUpdateTime": "2021-06-28T09:09:07.725+03:00",
  "ValueAlarmState": [],
  "SensorState": 0,
  "SensorHealth": "WDExpried",
  "SensorLink": "Google",
  "SensorLinkName": "https://www.google.com"
}
},
],
"Success": true,
"AuthorizationCode": 0,
"Messages": [],
"JsonPath": ""
}

```

Content data description:Instance of [SensorPropertiesResponse](#)**Possible properties descriptions:**

- Coordinates - coordinates of sensor on GEOvis map. Type: [GeoLocation](#)
- CoordinatesInWGS84 - coordinates of sensor on GEOvis map in WGS 84. Type: [GeoLocation](#)
- Coordinate System - coordinates system of sensor coordinates. Type: [Coordinates-Systems](#)
- Color - HEX color string. Type: *string*
- UseColorToMap – flag of using sensor's color as default color. Type: *boolean*
- Offset - last valid offset. Type: [SensorData](#)
- AxisOffset - last axis valid offset. Type: [SensorData](#)
- Reference - last valid reference. Type: [SensorData](#)
- AxisReference - last valid axis reference. Type: [SensorData](#)
- ValueAlarmState -
- SensorState - state of sensor. Type: [SensorData](#)
- SensorHealth - watchdog state of sensor. Type: *string*
- LastRawValue - last sensor measured RAW value. Type: [SensorData](#)
- Unit - unit of sensor measurements. Type: [PhysicalUnit](#)
- Sensortype - type of sensor (type of device is case of Virtual Sensor). Type: [SensorCategory](#)
- CustomerName - name given to sensor by customer. Type: *string*
- LastUpdateAt - time of last value. Type: *string presentation of DateTime*.
- Param1-5 - params 1 - 5. Type: *string*.
- Distance - distance. Type: [4.19DistanceModel](#)
- PublicFlag - flag of availability (by protection level) of sensor. Type: *boolean*.
- Sensorlink – sensor link. Type: *string*.
- Sensorlinkname – sensor's link name. Type: *string*.

04.12.2025

2.8. Sync sensors properties request

Synchronize properties of sensors.

Request URL

<https://geovis.io/api/rest/sync/{projectId:int}>

Minimum access level required

READ WRITE

Request type

POST

Request payload

Instance of [SyncRequest](#). Required properties: Databaseld, [SyncType](#) (must be **0**), SyncSensors.

Response

See [General response description](#)

04.12.2025

2.9. Sync sensors data request

Synchronize data of sensors.

Request URL

<https://geovis.io/api/rest/sync/{projectId:int}>

Minimum access level required

READ WRITE

Request type

POST

Request payload

Instance of [SyncRequest](#). Required properties: Databaseld, [SyncType](#) (must be 1), SyncData.

Response

See [General response description](#)

Axis data calculation

For sensor types of Prism, Theodolite, GPS, Position the server will automatically (re)calculate-AXIS values right after a successful sync of RAW values if the sensor has an axis assignment in GEOvis.

Before saving the data (SyncType = 1) it is checked whether the specified sensors exist in the specified database. If none of the sensors from "SyncData" exists, the request is not executed.

04.12.2025

2.10. Change sensors data state request

Synchronize data of sensors.

Request URL

<https://geovis.io/api/rest/changeStates/{projectId:int}>

Minimum access level required

READ WRITE

Request type

POST

Request payload

Instance of [SyncStatesPayload](#).

Response

See [General response description](#)

2.11. Delete sensors data request

Synchronize data of sensors.

Request URL

<https://geovis.io/api/rest/delete/{projectId:int}>

Minimum access level required

READ WRITE

Request type

POST

Request payload

Instance of [DeleteSensorsDataPayload](#).

Response

See [General response description](#)

04.12.2025

2.12. Get views of project request

Receive all views for project.

Request URL

<https://geovis.io/api/rest/views/{projectId:int}>

Minimum access level required

READ ONLY

Request type

GET

Request payload

No payload needed.

Response

Sample:

```
{
  "Data": [
    {
      "Id": "view_id_string",
      "Name": "Test View"
    },
    {
      "Id": "view_2_id",
      "Name": "Test 2"
    }
  ],
  "Success": true,
  "AuthorizationCode": 0,
  "Messages": [],
  "JsonPath": ""
}
```

Content data description:

Data is array of [ProjectViewObserveInfo](#)

04.12.2025

2.13. Get project view's info request

Receive information about project view.

Request URL

<https://geovis.io/api/rest/view/{projectId:int}/{viewId}>

Minimum access level required

READ ONLY

Request type

GET

Request payload

No payload needed.

Response

Sample:

```
{
  "Data": {
    "Id": "view_id_string",
    "Name": "Test View",
    "IsPublic": true,
    "Sensors": [
      {
        "SensorFullId": "DatabaseID.SensorID",
        "Id": "101",
        "CustomerName": "101a"
      },
      {
        "SensorFullId": "DatabaseID.SensorID",
        "Id": "102",
        "CustomerName": "102b"
      }
    ]
  },
  "Success": true,
  "AuthorizationCode": 0,
  "Messages": [],
  "JsonPath": ""
}
```

Content data description:

Data is an instance of [ProjectViewInfo](#)

04.12.2025

2.14. Add sensors to view request

Receive information about project view.

Request URL

<https://geovis.io/api/rest/addSensorsToView/{projectId:int}/{viewId}>

Minimum access level required

READ WRITE

Request type

POST

Request payload

Instance of [BaseSensorsPayload](#).

Sample:

```
{
  "SensorsFullIds": ["DatabaseID.SensorID", "DatabaseID.SensorID", "DatabaseID.SensorID"],
  "DatabaseSensors": [{
    "DatabaseId": "Project_database_ID_1",
    "SensorsIds": ["SensorId_1", "SensorId_2"]
  }, {
    "DatabaseId": "Project_database_ID_2",
    "SensorsIds": ["SensorId_3", "SensorId_4"]
  }]
}
```

Response

Sample:

```
{
  "Data": [],
  "Success": true,
  "AuthorizationCode": 0,
  "Messages": [],
  "JsonPath": ""
}
```

Content data description:

In case of good response data will be empty. If some problems occur during adding sensors all of them will be returned as array of string.

Sensor will not be added to the view if it's already added or if sensor has no coordinates.

04.12.2025

2.15. Remove sensors from view request

Receive information about project view.

Request URL

<https://geovis.io/api/rest/removeSensorsFromView/{projectId:int}/{viewId}>

Minimum access level required

READ WRITE

Request type

POST

Request payload

Instance of [BaseSensorsPayload](#).

Sample:

```
{
  "SensorsFullIds": ["DatabaseID.SensorID", "DatabaseID.SensorID", "DatabaseID.SensorID"],
  "DatabaseSensors": [{
    "DatabaseId": "Project_database_ID_1",
    "SensorsIds": ["SensorId_1", "SensorId_2"]
  }, {
    "DatabaseId": "Project_database_ID_2",
    "SensorsIds": ["SensorId_3", "SensorId_4"]
  }]
}
```

Response

Sample:

```
{
  "Data": [],
  "Success": true,
  "AuthorizationCode": 0,
  "Messages": [],
  "JsonPath": ""
}
```

Content data description:

In case of good response data will be empty. If some problems occur during removing sensors all of them will be returned as array of string.

04.12.2025

2.16. Get logbook labels request

Receive all labels defined for the specified project.

Request URL

<https://geovis.io/api/rest/getLabels/{projectId:int}>

Minimum access level required

READ ONLY

Request type

GET

Request payload

No payload needed.

Response

Sample:

```
{
  "Data": [
    {
      "Id": "b71b2f3a-7f8a-4a32-9c32-b0e3d92a5d11",
      "Name": "Crack"
    },
    {
      "Id": "f8af2d51-9ab0-4e61-9b43-6c26f11bdb2c",
      "Name": "Tunnel"
    }
  ],
  "Success": true,
  "AuthorizationCode": 0,
  "Messages": [],
  "JsonPath": ""
}
```

Content data description:

Data is an instance of [LabelResponse](#).

04.12.2025

2.17. Sync logbook labels request

Receive all labels defined for the specified project.

Request URL

<https://geovis.io/api/rest/syncLabels/{projectId:int}>

Minimum access level required

READ WRITE

Request type

POST

Request payload

Instance of [LabelPayload](#).

Sample:

```
{
  "Labels": [ "Label_name_1", "Label_name_2", "Label_name_2" ]
}
```

Response

See [General response description](#)

Content data description:

In case of good response data will be empty. If some problems occur during adding labels all of them will be returned as array of string.

If some labels already exist in the project, they are silently ignored.

04.12.2025

2.18. Get logbook request

Receive all logbook which match the filter.

Request URL

<https://geovis.io/api/rest/getLogbook/{projectId:int}>

Minimum access level required

READ ONLY

Request type

POST

Request payload

Instance of [LogbookFilter](#).

Sample:

```
{
  "TimePeriod": "00-00-14 00:00:00",
  "StartDate": "2020-11-24T09:09:07.725+02:00",
  "EndDate": "2022-11-28T09:09:07.725+02:00",
  "Class": [0, 5, 9],
  "Author": ["6863a5d00017d9f0444fb3cf"],
  "Label": ["Label_name_1", "Label_name_2"],
  "Confirmed": 0,
  "Public": 0
}
```

Response

Sample:

```
{
  "Id": "6863a5d00017d9f0444fb3cf ",
  "AuthorId": "6863a5d00017d9f0444fb3cf",
  "Date": "2020-11-24T09:09:07.725+02:00",
  "Class": 5,
  "Labels": [
    ["Label_name_1", "Label_name_2"]
  ],
  "Description": "Description",
  "Comment": "Comment",
  "Public": true,
  "Confirmed": null
}
```

Content data description:

Data is an array of [LogbookResponse](#).

04.12.2025

2.19. Sync logbook request

Create new logbook for the specified project.

Request URL

<https://geovis.io/api/rest/syncLogbook/{projectId:int}>

Minimum access level required

READ WRITE

Request type

POST

Request payload

Array of [LogbookPayload](#).

Sample:

```
[
  {
    "Date": "2020-11-24T09:09:07.725+02:00",
    "Labels": ["Label_name_1", "Label_name_2"],
    "Description": "Description",
    "Comment": "Comment",
    "Public": True
  },
  {
    "Date": "2020-11-25T19:09:07.725+02:00",
    "Labels": ["Label_name_1"],
    "Description": "Description",
  }
]
```

Response

See [General response description](#)

Content data description:

In case of good response data will be empty. If some problems occur during adding logbook all of them will be returned as array of string.

04.12.2025

2.20. Get inspection objects of project

Return all inspection objects, created in the project.

Request URL

<https://geovis.io/api/rest/{projectId}/inspections/objects>

Minimum access level required

READ

Request type

GET

Request payload

No payload needed.

Response

Sample:

```
{
  "DatabaseId": "Inspection_DB_ID",
  "DatabaseName": "Inspection_1",
  "InspectionObjects": [
    {
      "Uid": "123",
      "Name": "Inspection object name"
    }
  ]
}
```

Content data description:

Data is an array of [InspectionDatabaseModel](#)

04.12.2025

2.21. Get inspection object info

Return info about of inspection object.

Request URL

<https://geovis.io/api/rest/{projectId}/inspections/object/{alias}/{uid}>

Minimum access level required

READ

Request type

GET

Request payload

No payload needed.

Response

Sample:

```
{
  "LatestAssessmentRating": 3,
  "LatestAssessmentDate": "02.12.2025",
  "Name": "Test inspection with files",
  "Uid": "123",
  "Color": "#ffff00",
  "Points": []
}
```

Content data description:

Data is an array of [InspectionObjectInfo](#)

04.12.2025

3. Used enums descriptions

Some data properties in response are values of a specific enum. Here described all enums used in GEOvis REST API.

3.1. AuthorizationCode

Purpose: used in every response to return result code of request

```
enum AuthorizationCode
{
    OK = 0, // User is authorized and has required permissions
    AuthorizationFailed = 1, // User authorization failed (expired)
    NoReadPermissions = 2, // User is authorized, but have not read permissions
    NoWritePermissions = 3, // User has no write permissions
    ElementNotFound = 4, // The expected element not found
    UnexpectedError = 5, // Unexpected error
    ImpossibleOperation = 6, // Impossible operation, result contains description
    Cancelled = 7, // Operation cancelled
    NotImplemented = 8, // Operation not implemented
    NotInitialized = 9, // The expected element is not initialized
    EmptyRequest = 10, // Empty request
    Failed = 11, // Processing failed
    ValidationFailed = 12, // Fail validation of object,
    TooBigResponse = 13, // The response is too big for this request
    LicensingFailed = 14, // GEOvis4.0 licensing failed
    TimeoutException = 15 // No connection to the remote endpoint
}
```

3.2. ProjectAutomaticState

Purpose: used for displaying automatic state of project

```
enum ProjectAutomaticState
{
    Ok = 0, // Project is OK
    InterventionRequired = 1, // Project has value alarm
    FunctionalAlarm = 2, // Project has functional alarm
}
```

3.3. ProjectManualState

Purpose: defines possible manual states of project

```
enum ProjectManualState
{
    Running = 0, // Normal project state
    Finished = 1, // Project is finished
    OnHold = 2 // Project is on hold, but still on GEOvis System
}
```

04.12.2025

3.4. SensorCategory

Purpose: used for defining physical type of sensor or device type in case of virtual sensor

```
enum SensorCategory
{
    Unknown = 0,
    Prism = 1, // Prism
    Theodolite = 2, // Theodolite
    BH_Extensometer = 3, // Extensometer
    Extensometer = 4, // Extensometer
    Inclinator = 6, // Inclinator
    BH_Temperature = 7, // Temperature
    Temperature = 8, // Temperature
    Piezometer = 9, // Piezometer
    Air_Pressure = 10, // Pressure
    Wind_Speed = 11, // Speed
    Solar_Radiation = 12, // Solar radiation
    Distance = 13, // Distance
    Pressure_Cell = 14, // Load cell
    Load_Cell = 15, // Load cell
    Voltage = 16, // Voltage
    VirtualSensor = 17, Virtual Sensor
    VibrationSensor = 18, // Vibration sensor
    InclinatorChainPosition = 19, // Inclinator chain position
    AgmsVirtualSensor = 20, // Virtual sensor
    HydrostaticLevel = 21, // Hydrostatic level
    CurrentLoop = 22, // Current
    Counter = 23, // Counter
    Noise = 24, // Noise
    Rainfall = 25, // Rainmeter
    GH = 26, // Track control GH
    V = 27, // Track control Verwindung
    PfV = 28, // Track control PfV
    Settlement = 29, // Settlement
    GPS = 30, // GPS
    Flow = 31, // Flow
    Volume = 32, // Volume
    Crackmeter = 33, // Crackmeter
    PfH = 34, // PfH
    pH = 35, // pH
    Turbidity = 36, // Turbidity (NTU)
    Leveling = 37, // Tachymeter leveling
    Orientation = 38, // Tachymeter Orientation
    GasConcentration = 39, // Gas concentration
    Attenuation = 40, // Attenuation
    Power = 41, // Power
    Stoke = 42, // Stoke
    ThermalConductivity = 43, // Thermal conductivity
    PorePressure = 44, // Pore pressure
    Frequency = 45, // Frequency
    RelativeHumidity = 46, // Humidity Relative
    ThermalResistivity = 47, // Thermal Resistivity
    WaterContent = 48, // Water content
    Time = 49, // Time
    Dimentionless = 50, // Dimensionless
    NormalizedTemperature = 51, // Normalized Temperature
    DTS_Temperature = 52, // DTS Temperature
    DTS_Stoke_Antistoke = 53, // DTS_Stoke/Antistoke
    DTS_Attenuation = 54, // DTS_Attenuation
    DTS_Position = 55,
    TM = 56 // for DTS
    StrainGauge = 57,
    Resistance = 58,
    DTS_ThermalConductivity = 59,
```

04.12.2025

```

    MST = 60,
    SB = 61,
    Variance = 62,
    MST_SB = 63,
    Gauge = 64,
    ElectricalConductivity = 65,
    SignalStrength = 66,
    SignalNoiseRatio = 67,
    ErrorRate = 68,
    Acceleration = 69,
    ReactiveEnergy = 70,
    Illuminance = 71,
    PartsPer = 72,
    Weight = 73,
    WeightPerTime = 74,
    OpticalLoss = 75,
    FibreOpticTemperature = 76,
    Intensity = 77,
    Gateway = 78,
    Position = 79,
    Magnitude = 80,
    Height = 81, // Height
    Modem = 82, // Modem
  }

```

3.5. PhysicalUnit

Purpose: describes all possible units of sensors measurements

```

enum PhysicalUnit
{
    Undefined = 0,
    Meter = 1,
    Millimeter = 2,
    Pascal = 3,
    Radiant = 4,
    Degree = 5,
    GON = 6,
    Kelvin = 7,
    Celsius = 8,
    Milliampere = 9,
    Ohm = 10,
    Hectopascal = 11,
    Megapascal = 12,
    MeterPerSecond = 13,
    WattPerSquareMeter = 14,
    Newton = 15,
    Volt = 16,
    M3ps = 17, // cubic meter per second
    M3ph = 18, // cubic meter per hour
    Percent = 19,
    Promille = 20,
    Kilometer = 21,
    Herz = 22,
    MillimeterPerSecondInQuad = 23, // millimeter / second squared
    MillimeterPerSecond = 24,
    Second = 25,
    hPa = 26, // Hectopascal
    KiloNewton = 27,
    m3ps = 28, // Flow cubic meter per second
    m3ph = 29, // Flow cubic meter per hour
    m3pm = 30, // Flow cubic meter per minute
    NTU = 31, // Turbidity unit
}

```

04.12.2025

pH = 32,
 M3 = 33, // cubic meter
 MillimeterPerMeter = 34,
 dB = 35,
 mmph = 36, // millimeter per hour
 Centimeter = 37,
 MeterAboveSeaLevel = 38,
 LiterPerHour = 39,
 kOhm = 40,
 MOhm = 41,
 KilometerPerHour = 42,
 mgon = 43,
 cgon = 44,
 KiloPascal = 45,
 mlnm = 46, // normal milliliter per minute
 VolumePercent = 47,
 Amper = 48,
 Watt = 49,
 pWperpW = 50, // picoWatt per picoWatt
 WpmpK = 51, // Thermal Conductivity unit
 RHPercent = 52, // HumidityRel unit
 CmpW = 53, // Thermal Resisitivity unit
 WC = 55, // Water content
 MilliSecond = 56,
 Minute = 57,
 Hour = 58,
 Day = 59,
 VPM = 60, // Volumetric percentage
 PromileVPDB = 61,
 PromileVSMOW = 62,
 PromileVCDT = 63,
 MilliBar = 64,
 KiloHerz = 65,
 MegaHerz = 66,
 CcmpW = 67, // (K*cm)/W
 CM3STPW = 68, // cm3STP/cm3STP
 NanoSecond = 69,
 KiloWatt = 70,
 VDC = 71,
 VAC = 72,
 MilliVolt = 73,
 NTemp = 74, // (m*mK)/(ln(s)*W)
 Decimeter = 75,
 Microstrain = 76,
 Bar = 77,
 SiemensPerMeter = 78,
 MilliSiemensPerMeter = 79,
 MilliSiemensPerCentimeter = 80,
 MillimeterH2O = 81,
 MeterH2O = 82,
 dBm = 83,
 mmPerH = 84,
 cmPerH = 85,
 dmPerH = 86,
 mPerH = 87,
 mmPerDay = 88,
 cmPerDay = 89,
 dmPerDay = 90,
 mPerDay = 91,
 mmPerWeek = 92,
 cmPerWeek = 93,
 dmPerWeek = 94,
 mPerWeek = 95,
 mmPerMonth = 96,
 cmPerMonth = 97,

04.12.2025

```

    dmPerMonth = 98,
    mPerMonth = 99,
    mmPerYear = 100,
    cmPerYear = 101,
    dmPerYear = 102,
    mPerYear = 103,
    MeterPerSecondInQuad = 104,
    VoltAmpereReactiveHour = 105,
    KiloVoltAmpereReactiveHour = 106,
    Lux = 107,
    PartsPerHundert = 108,
    PartsPerMillion = 109,
    PartsPerBillion = 110,
    PartsPerTrillion = 111,
    Ton = 112,
    KiloGramm = 113,
    Gramm = 114,
    MilliGramm = 115,
    MilliliterPerMinute = 116,
    Milliliter = 117,
    MicroSiemensPerMeter = 118,
    GrammPerHour = 119,
    GigaHerz = 120,
    dBPerKm = 121,
    Liter = 122
  }

```

3.6. SensorState

Purpose: defines possible states of sensor

```

enum SensorState
{
    Ok = 0,
    Inactive = 1,
    CalculationDisabled = 2
}

```

3.7. ValuesContent

Purpose: defines options for count of values in sensor data

```

enum ValuesContent
{
    Values = 0, // all raw values
    Value1 = 1, // raw value 1
    Value2 = 2, // raw value 2
    Value3 = 3, // raw value 3
    Axes = 4, // all axis values
    Axis1 = 5, // axis value 1
    Axis2 = 6, // axis value 2
    Axis3 = 7 // axis value 3
}

```

3.8. RequestedValueType

Purpose: defines options for relativity type of sensor value

04.12.2025

enum RequestedValueType

```
{  
    RAW = 0, // base value  
    ABSOLUT = 1, // base value + offset value  
    RELATIVE = 2 // base value + offset value - reference value  
}
```

3.9. DistanceUnit

Purpose: defines options for distance unit of sensor

enum DistanceUnit

```
{  
    Meter = 0, // distance measure in meters  
    Kilometer = 1, // distance measure in kilometers  
}
```

04.12.2025

3.10. SensorProperties

Purpose: defines available sensor's properties options to be collected with GEOvis REST API

```
enum SensorPropertiesREST
{
    SensorType = 0, // Type of sensor (Type of Device in case of virtual sensor)
    CustomerName = 1, // Sensor's customer name
    Unit = 2, // Unit of sensor's measurements
    PublicFlag = 3, // Flag shows if sensor is public
    Coordinates = 4, // Map coordinates of sensor
    CoordinatesSystem = 5, // Map coordinates system
    Color = 6, // Sensor color
    UseColorToMapFlag = 7, // use Sensor color for Map
    Distance = 8, // Distance
    Param1 = 9, // Sensor's custom param 1
    Param2 = 10, // Sensor's custom param 2
    Param3 = 11, // Sensor's custom param 3
    Param4 = 12, // Sensor's custom param 4
    Param5 = 13, // Sensor's custom param 5
    LastValue = 14, // Last RAW value of sensor
    LastAxisValue = 15, // Last AXIS value of sensor
    Offset = 16, // Last valid RAW offset
    AxisOffset = 17, // Last valid AXIS offset
    Reference = 18, // Last valid RAW reference
    AxisReference = 19, // Last valid AXIS reference
    LastUpdateTime = 20, // Time when sensor get last value
    ValueAlarmState = 21, // Info about alarm of the sensor
    SensorState = 22, // Sensor's measurement state
    SensorHealth = 23, // Sensor's WD health
    SensorLink = 24, // Sensor full Link
    SensorLinkName = 25, // Name of Link will be displayed in Sensor Popup
    CoordinatesInWGS84 = 26, // Map coordinates of sensor in WGS 84
    LocH = 27, // Loga Hight coordinates for NAGRA (depends on Angle an Radius)
    LocY = 28, // Local Y-Axis coordinates for NAGRA (depends on Angle an Radius)
    TunnelMeter = 29, // TunnelMeter coordinates of local coordinate system for NAGRA
    Angle = 30, // Angle coordinates of local coordinate system for NAGRA
    Radius = 31, // Radius coordinates of local coordinate system for NAGRA
    LocX = 32, // Local X-Axis coordinates for NAGRA (depends on Angle an Radius)
}
```

3.11. CoordinatesSystems

Purpose: define all possible coordinates systems

```
enum CoordinateSystems
{
    WGS84 = 0, // WGS84 GPS (EPSG:4326)
    CH1903 = 1, // LV03 (EPSG:21781)
    CH1903plus = 2, // CH1903+ / LV95 (EPSG:2056)
    DB_REF_GK3 = 3, // 3-degree Gauss-Kruger zone 3 (E-N) (EPSG:5683)
    EPSG_3857 = 4, // WGS84 Pseudo Mercator (EPSG:3857)
    EPSG_31466 = 5, // 3-degree Gauss-Kruger zone 2 (EPSG:31466)
    EPSG_31467 = 6, // 3-degree Gauss-Kruger zone 3 (EPSG:31467)
    EPSG_31468 = 7, // 3-degree Gauss-Kruger zone 4 (EPSG:31468)
    EPSG_31469 = 8, // 3-degree Gauss-Kruger zone 5 (EPSG:31469)
    EPSG_3949 = 9, // RGF93 / CC49 (EPSG:3949)
    DB_REF_GK2 = 10, // 3-degree Gauss-Kruger zone 2 (E-N) (EPSG:5682)
    DB_REF_GK4 = 11, // 3-degree Gauss-Kruger zone 4 (E-N) (EPSG:5684)
    DB_REF_GK5 = 12, // 3-degree Gauss-Kruger zone 5 (E-N) (EPSG:5685)
    EPSG_9932 = 13, // DB_REF2016 zone 2 (EPSG:9932)
    EPSG_9933 = 14, // DB_REF2016 zone 3 (EPSG:9933)
    EPSG_9934 = 15, // DB_REF2016 zone 4 (EPSG:9934)
}
```

04.12.2025

```

    EPSG_9935 = 16, // DB_REF2016 zone 5 (EPSG:9935)
    EPSG_31276 = 17, // MGI/Balkans zone 6 (EPSG:31276)
    EPSG_5950 = 18, // NTM zone 10 + NN2000 height (EPSG:5950)
    EPSG_3908 = 19, // MGI 1901/Balkans zone 6 (EPSG:3908)
    EPSG_31266 = 20, // MGI / 3-degree Gauss zone 6 (EPSG:31266)
    ETRS_89 = 21, // Hamburg / Gauss-Krüger-LS 320 (ETRS 89)
    EPSG_25832 = 22, // UTM zone 32N (EPSG:25832)
    EPSG_7854 = 23, // MGA Zone 54 (GDA2020) (EPSG:7854)
    EPSG_7855 = 24, // MGA Zone 55 (GDA2020) (EPSG:7855)
    EPSG_7856 = 25, // MGA Zone 56 (GDA2020) (EPSG:7856)
    EPSG_31251 = 26, // MGI (Ferro) / Austria GK West Zone (EPSG:31251)
    EPSG_31252 = 27, // MGI (Ferro) / Austria GK Central Zone (EPSG:31252)
    EPSG_31253 = 28, // MGI (Ferro) / Austria GK East Zone (EPSG:31253)
    EPSG_4093 = 29, // DKTm1 (EPSG:4093)
    EPSG_4094 = 30, // DKTm2 (EPSG:4094)
    EPSG_4095 = 31, // DKTm3 (EPSG:4095)
    EPSG_4096 = 32, // DKTm4 (EPSG:4096)
    EPSG_5846 = 33, // SWEREF99 12 00 + RH2000 (Gothenburg area) (EPSG:5846)
    EPSG_5850 = 34, // SWEREF99 18 00 + RH2000 (Stockholm area) (EPSG:5850)
    EPSG_3011 = 35 // SWEREF99 18 00 (Stockholm area) (EPSG:3011)
}
enum CoordinateSystems
{
    WGS84 = 0, // WGS84 GPS (EPSG:4326)

    CH1903 = 1, // LV03 (EPSG:21781)

    CH1903plus = 2, // (EPSG:2056) CH1903+ or LV95 coordinate system

    DB_REF_GK3 = 3, // 3-degree Gauss-Kruger zone 3 (E-N)

    EPSG_3857 = 4, // (EPSG:3857) WGS 84 / Pseudo-Mercator -- Spherical Mercator, Google Maps, Open-StreetMap, Bing, ArcGIS, ESRI

    EPSG_31466 = 5, // 3-degree Gauss-Kruger zone 2

    EPSG_31467 = 6, // 3-degree Gauss-Kruger zone 3

    EPSG_31468 = 7, // 3-degree Gauss-Kruger zone 4

    EPSG_31469 = 8, // 3-degree Gauss-Kruger zone 5

    EPSG_3949 = 9, // (EPSG:3949) RGF93 / CC49

    DB_REF_GK2 = 10, // (EPSG:5682) 3-degree Gauss-Kruger zone 2 (E-N)
    DB_REF_GK4 = 11, // (EPSG:5684) 3-degree Gauss-Kruger zone 4 (E-N)

    DB_REF_GK5=12, // (EPSG:5685) 3-degree Gauss-Kruger zone 5 (E-N)

    EPSG_9932=13, // (EPSG:9932) DB_REF2016 zone 2

    EPSG_9933=14, // (EPSG:9933) DB_REF2016 zone 3

    EPSG_9934=15, // (EPSG:9934) DB_REF2016 zone 4

    EPSG_9935=16, // (EPSG:9935) DB_REF2016 zone 5

    EPSG_31276=17, // (EPSG:31276) MGI/Balcans zone 6

    EPSG_5950=18, // (EPSG:5950) NTM zone 10 + NN2000 height

    EPSG_3908=19, // (EPSG:3908) MGI 1901/Balkans zone 6

    EPSG_31266=20, // (EPSG:31266) MGI/3-degree Gauss zone 6

```


04.12.2025

}

3.12. SyncType

Purpose: defines type of synchronization

```
enum SyncType
{
    SensorsDescriptions = 0, // sync sensor's properties
    SensorsData = 1 // sync sensor's data
}
```

3.13. SyncDataOverwriteType

Purpose: defines handling data duplicates

```
enum SyncDataOverwriteType
{
    Overwrite = 0, // Overwrite existing data by new data
    KeepOld = 1, // New data will be ignored
    WriteNewAsDuplicate = 2, // All data will be stored, but incoming data will have duplicate status
    ChangeOldToDuplicate = 3 // All data will be stored, but existing data will have duplicate status
}
```

04.12.2025

3.14. ExportDataDensityType

Purpose: defines type of exporting data

Note: value should be passed as string

```
enum ExportDataDensityType
{
    "OriginalData", // Raw data without offset and reference
    "Buckets",
    "ReducedData" // Reduced / Interpolated Data
}
```

3.15. ExportDataPeriod

Purpose: defines export data period

Note: value should be passed as string

```
enum ExportDataPeriod
{
    "min15", //15 minutes
    "min30", //30 minutes
    "hour1", //1 hour
    "hour2", //2 hours
    "hour4", //4 hours
    "hour6", //6 hours
    "hour12", //12 hours
    "hour24" //24 hours
}
```

3.16. BucketReportValueKind

Purpose: defines which values should be taken in every bucket

Note: value should be passed as string

```
enum BucketReportValueKind
{
    "MinMax", // Return only min/max values
    "Median" // Return only median value
}
```

04.12.2025

3.17. BucketIntervalType

Purpose: defines interval of data

Note: value should be passed as string

```
enum BucketIntervalType
{
    "Default", //Default
    "Every15Minutes", //15 minutes
    "Every30Minutes", //30 minutes
    "EveryHour", //1 hour
    "Every2Hours", //2 hours
    "Every4Hours", //4 hours
    "Every8Hours", //8 hours
    "Every12Hours", //12 hours
    "Every1Day" //24 hours
}
```

3.18. InterpolationAlgorithmMode

Purpose: defines interpolation algorithm mode

```
enum InterpolationAlgorithmMode
{
    Default = 0, //Normal mode
    Fast = 1, //Fast mode,
    NormalMedian = 2, //Normal mode (median)
    NormalMedianBackwards = 3, Normal mode (median / only go T/2 backwards)
}
```

3.19. MongoDataState

Purpose: defines state of sensor's data

```
enum MongoDataState
{
    OK = 0, // Measured data is correct
    CALCULATION_DISABLED= 2, //Fast mode
    OUT_OF_RANGE=6, //Out of range
    DUPLICATE_OVERWRITE=11 //duplicate
}
```

3.20. SensorMode

Purpose: used as sync Flag in DataItem

```
enum SensorMode
{
    Unknown = 0,
    Reference = 1, // "R"
    Offset = 2, "O"
}
```

3.21. RequestedValueType

Purpose: defines what time will be used in data

enum RequestedValueTime

```
{  
    Timeslot = 0, // Timeslot will be returned  
    ExactTime = 1, //Exact time will be returned  
}
```

04.12.2025

3.22. RequestedValueTimeFormat

Purpose: defines format of time string in data entry

```
enum RequestedValueTimeFormat
{
    Default = 0, // Time format will be yyyy-MM-dd HH:mm:ss
    WithMilliseconds = 1, //Time format will be yyyy-MM-dd HH:mm:ss.fff
}
```

3.23. LogbookType

Purpose: defines possible type of logbook

```
enum LogbookType
{
    Manual = 0, // Manual entry created by user
    Alarm = 1, // Generic alarm entry
    Watchdog = 2, // System watchdog entry
    ImportTask = 3, // Task watchdog entry
    AGMSNotifications = 5, // FA4: AGMS Notifications
    SensorWatchdog = 6, // Sensor Watchdog
    ChainWatchdog = 7, // Chain Watchdog
    LoRa = 8, // FA5: LoRa Notifications
    TachymeterSystemNotifications = 9, // FA1: Tachymeter System Notifications
    TrackControlSystemNotifications = 10, // FA2: TrackControl System Notifications
    GeotechnicSensorsNotifications = 11, // FA3: Geotechnic Sensors Notifications
    TaskNotifications = 12, // FA6: Task Notifications
    TestAlarm = 13, // Test alarm
    Service = 14, // Service messages
}
```

3.24. LogbookConfirmed

Purpose: defines confirmation status filter for logbook

```
enum LogbookConfirmed
{
    All = 0, // Do not filter by confirmation status
    Confirmed = 1, // Only confirmed entries
    NotConfirmed = 2 // Only not confirmed entries
}
```

3.25. LogbookPublic

Purpose: defines confirmation status filter for logbook

```
enum LogbookPublic
{
    All = 0, // Do not filter by public flag
    Public = 1, // Only public entries
    NotPublic = 2 // Only non-public entries
}
```

04.12.2025

4. Objects structures description

Some complex objects are using in GEOvis REST API requests and responses.

4.1. ProjectObserveModel

Structure contains observe info about project.

JSON document sample:

```
{
  "Id": 1,
  "Name": "Project 1",
  "Number": "PR_1"
}
```

Fields description:

- **"Id"** - unique project ID. Type: *Int32*.
- **"Name"** - project name. Type: *string*.
- **"Number"** - project number. Type: *string*.

4.2. DatabaseModel

Structure contains observe info about database

JSON document sample:

```
{
  "Id": 1_1,
  "Name": "Database 1"
}
```

Fields description:

- **"Id"** - database ID. Unique in GEOvis system. All communications with databases using this data. Type: *string*.
- **"Name"** - database customer name. Unique in project. Type: *string*.

4.3. UserModel

Structure contains observe info about user.

JSON document sample:

```
{
  "FullName": "User, Name",
  "Email": "user@email.com",
  "Phone": "+123456789"
}
```

Fields description:

- **"FullName"** - name of user. Type: *string*.
- **"Email"** - E-mail address of user. Type: *string*.
- **"Phone"** - phone number of user. Type: *string*.

04.12.2025

4.4. GeoLocation

Structure contains geo coordinate point

JSON document sample:

```
{
  "Longitude": 8.47448229789734,
  "Latitude": 47.440043819152663,
  "Height": 0.0
}
```

Fields description:

- **"Longitude"** - longitude of geopoint. Type: *double*.
- **"Latitude"** - latitude of geopoint. Type: *double*.
- **"Height"** - height of geopoint. Type: *double*.

4.5. ProjectInfoModel

Structure contains detailed info about project.

JSON document sample:

```
{
  "Id": 1,
  "Health": [0, 1, 2],
  "CoordinateSystem": "1",
  "CustomerCompany": "Customer Company Name",
  "Location": {
    "Longitude": 8.47448229789734,
    "Latitude": 47.440043819152663,
    "Height": 0.0
  },
  "Leader": {
    "FullName": "Leader, Name",
    "Email": "leader@email.com",
    "Phone": "+123456789"
  },
  "Databases": [
    {
      "Id": "1_1",
      "Name": "DB_1"
    },
    ...
    {
      "Id": "1_999",
      "Name": "DB_999"
    }
  ],
  "State": 0
}
```

Fields description:

- **"Id"** - unique project ID. Type: *Int32*.
- **"Health"** - array with automatically detected project states. Type: *Array of [ProjectAutomaticState](#)*
- **"CoordinateSystem"** - coordinate system of the project. Type: [CoordinateSystems](#).
- **"CustomerCompany"** - name of project's customer company. Type: *string*.
- **"Location"** - coordinates of project base point in WGS84. Type: [GeoLocation](#)
- **"Leader"** - info about leader of the project. Type: [UserModel](#)
- **"Databases"** - array of all databases in project. Type: *Array of [DatabaseModel](#)*

04.12.2025

- **"State"** - project manual state. Type: [ProjectManualState](#)

4.6. SensorsSelectionFilter

Structure contains filters for sensors selection. All filters' conditions merged with AND operator.

JSON document sample:

```
{
  "DatabaseId": "1_1",
  "NameSearch": "",
  "SensorType": 0,
  "Unit": 1,
  "SensorState": 2
}
```

Fields description:

- **"DatabaseId"** - specify ID of database. Can be skipped. Type: *string*.
- **"NameSearch"** - sensors names of returned sensors should match this pattern. Can be skipped. Type: *string*.
- **"SensorType"** - only sensors of this type will be returned. Can be skipped. Type: *SensorCategory*
- **"Unit"** - only sensors with measurements in this unit will be returned. Can be skipped. Type: [PhysicalUnit](#)
- **"SensorState"** - only sensors with this state will be returned. Can be skipped. Type: [SensorState](#)

4.7. SensorObserveModel

Structure contains observe data about sensor.

JSON document sample:

```
{
  "SensorFullId": "DatabaseID.SensorID",
  "Id": "",
  "CustomerName": 0
}
```

Fields description:

- **"SensorFullId"** - full Id of sensor (DatabaseID.SensorID). Should be used in each request for collecting sensors data. Type: *string*.
- **"Id"** - ID of sensor. Type: *string*.
- **"CustomerName"** - sensor's customer name. Type: *string*.

04.12.2025

4.8. SensorsDataFilter

Structure contains filters for sensor's data collecting. All filters' conditions merged with AND operator.

JSON document sample:

```
{
  "SensorsFullIds": ["DatabaseID.SensorID", "DatabaseID.SensorID", "DatabaseID.SensorID"],
  "DatabaseSensors": [{
    "DatabaseId": "Project_database_ID_1",
    "SensorsIds": ["SensorId_1", "SensorId_2"]
  }, {
    "DatabaseId": "Project_database_ID_2",
    "SensorsIds": ["SensorId_3", "SensorId_4"]
  }],
  "StartDate": "2021-06-28T09:09:07.725+02:00",
  "EndDate": "2021-06-30T09:09:07.725+02:00",
  "ExactTimeslot": "2021-06-28T09:09:07.725+02:00",
  "TimePeriod": "00-00-00 01:10:00",
  "ValueTo": 20,
  "ValueFrom": 2.5,
  "GetLastData": true,
  "RequestedValues": [1],
  "RequestedValueType": 2,
  "RequestedValueTime": 0,
  "RequestedValueTimeFormat": 0,
  "States": [0],
  "ExportDataOptions": {
    "DataDensityType": "OriginalData",
    "DataPeriod": "hour24",
    "AlgorithmMode": 0,
    "BucketValueKind": "MinMax",
    "BucketIntervalType": "EveryHour"
  }
}
```

Fields description:

- **"SensorsFullIds"** - data will be returned only for these sensors. FullId (DatabaseID.SensorID) of each sensor should be sent in this field. Can be skipped if "Database Sensors" field is set. Type: *array of string*.
- **"DatabaseSensors"** – data will be returned only for specified sensors of databases. This field is alternative way to set sensors for processing. Can be skipped if "SensorsFullIds" field is set. Type: array of [DatabaseSensors](#).
- **"TimePeriod"** - time range for data selecting. Dominates *StartDate* and *EndDate*. If needed to use *StartDate* and *EndDate* should be skipped or set to "00-00-00 00:00:00". Only available format is "yyyy-MM-dd HH:mm:ss". Every value, like years, days or minutes can be any integer value. It means that TimePeriod value "0-0-1 0-100-300" will be valid (Time range in this case will be 1 day 100 minutes 300 seconds or 1 day 1 hour 45 minutes). Can be skipped. Type: String.
- **"ExactTimeslot"** - exact time for a single timeslot selection. Dominates *StartDate* and *EndDate*. Type: *string presentation of DateTime*.
- **"StartDate"** - start date of time range for data selecting. Should be less or equal than *EndDate*. Make sense only if *TimePeriod* is skipped or zero. Can be skipped in case of *TimePeriod* is not skipped or zero otherwise can **not** be skipped. Type: *string presentation of DateTime*.
- **"EndDate"** - end date of time range for data selecting. Should be greater or equal than *StartDate*. Make sense only if *TimePeriod* is skipped or zero.. Can be skipped in case of *TimePeriod* is not skipped or zero otherwise can **not** be skipped. Type: *string presentation of DateTime*.

04.12.2025

- **"ValueFrom"** - only sensor's values greater or equal than this value will be returned. Can be skipped. Type: *double*.
- **"ValueTo"** - only sensor's values less or equal than this value will be returned. Can be skipped. Type: *double*.
- **"GetLastData"** - if set as "True" time range will be extended until last value of sensor will be found. If skipped means "False". Type: *boolean*.
- **"RequestedValues"** - specify values number of sensor's values. If skipped returns VALUE1. Type: *array of [ValuesContent](#)*
- **"RequestedValueType"** - specify relativity of returned sensor's values. If skipped returns RAW values. Type: *[RequestedValueType](#)*
- **"States"** – specify states of values which will be returned by request. Can be skipped. If skipped means [0]. Type: *[MongoDataState](#)*.
- **"ExportDataOptions"** – extended export options for using data stored in buckets. Type: *[ExtendedExportOptions](#)*.
- **"RequestedValueTime"** – export option for time which will be returned in response data. If skipped means 0. Type: *[RequestedValueTime](#)*.
- **"RequestedValueTimeFormat"** – export option for time format of response data. If skipped means 0. Type: *[RequestedValueTimeFormat](#)*

4.9. SensorData

Structure contains data for single timeslot.

JSON document sample:

```
{
  "Timeslot": "10.01.2021 11:12:33",
  "Values": [1.2,2.1,3.2]
}
```

Fields description:

- **"Timeslot"** - timeslot. Type: *string presentation of DateTime*.
- **"Values"** - values for timeslot. Type: *array of double*.

4.10. SensorDataModel

Structure contains all found data for 1 sensor.

JSON document sample:

```
{
  "SensorFullId": "DatabaseID.SensorID",
  "Data": [
    {
      "Timeslot": "10.01.2021 11:12:33",
      "Values": [1.2,2.1,3.2]
    },
    ...
    {
      "Timeslot": "10.01.2021 15:12:33",
      "Values": [0.2,1.1,-1.2]
    }
  ]
}
```

Fields description:

- **"SensorFullId"** - fullId of sensor (DatabaseID.SensorID). Type: *string*.

04.12.2025

- **"Data"** - all values collected for this sensor. Type: *array of* [SensorData](#)

4.11. ExtendedExportOptions

Structure with extended filters for using bucket mode of sensors data receiving

JSON document sample:

```
{
  "DataDensityType": "OriginalData",
  "DataPeriod": "hour24",
  "AlgorithmMode": 0,
  "BucketValueKind": "MinMax",
  "BucketIntervalType": "EveryHour"
}
```

Fields description:

- **"DataDensityType"** – kind of returned data. Type: [3.14ExportDataDensityType](#).
- **"DataPeriod"** – period of returned data for reduced / interpolated data . In case of other density type makes no effect to result. Type: [ExportDataPeriod](#).
- **"AlgorithmMode"** – kind of used data receiving algorithm. Only used if density is Reduced/Interpolated data. Type: [InterpolationAlgorithmMode](#).
- **"BucketValueKind"** – bucket type. Only used if density type is Bucket. Type: [Bucket-ReportValueKind](#).
- **"BucketIntervalType"** – bucket interval. Only used if density type is Bucket. Type: [BucketIntervalType](#).

4.12. SensorPropertiesPayload

Structure contains filters for collecting sensors properties.

JSON document sample:

```
{
  "SensorsFullIds": ["DatabaseID.SensorID", "DatabaseID.SensorID", "DatabaseID.SensorID"],
  "DatabaseSensors": [{
    "DatabaseId": "Project_database_ID_1",
    "SensorsIds": ["SensorId_1", "SensorId_2"]
  }, {
    "DatabaseId": "Project_database_ID_2",
    "SensorsIds": ["SensorId_3", "SensorId_4"]
  }],
  "RequestedParameters": [0, 1, 2, 3, 5, 6, 7, 8],
  "HumanReadable": false
}
```

Fields description:

- **"SensorsFullIds"** - data will be returned only for these sensors. FullId (DatabaseID.SensorID) of each sensor should be sent in this field. Can be skipped if "Database Sensors" field is set. Type: *array of string*.
- **"DatabaseSensors"** – data will be returned only for specified sensors of databases. This field is alternative way to set sensors for processing. Can be skipped if "SensorsFullIds" field is set. Type: *array of* [DatabaseSensors](#).
- **"RequestedParameters"** - array with all needed properties signatures to be collected. Can **not** be skipped. Type: *array of* [SensorProperties](#).
- **"HumanReadable"** – flag to return Sensor Type and Sensor Unit as human readable values instead of codes. Can be skipped. When skipped means "False". Type: *boolean value*.

04.12.2025

4.13. SensorPropertiesResponse

Structure contains all requested properties values of sensor

JSON document sample:

```
{
  "SensorFullId": "DatabaseID.SensorID",
  "Properties": {
    "SensorType": 13,
    "CustomerName": "",
    "Unit": 2,
    "PublicFlag": true,
    "Coordinates": {
      "Longitude": 8.2146318885368981,
      "Latitude": 48.84660510513914,
      "Height": 121.223
    },
    "CoordinatesSystem": 0,
    "Color": "#FAA3F8",
    "UseColorToMapFlag": false,
    "Distance": {
      "Value": 0.0,
      "Unit": 1
    },
    "Param1": "",
    "Param2": "",
    "Param3": "",
    "Param4": "",
    "Param5": "",
    "LastValue": "06.05.2021 13:50:00: 2671812,1056 ",
    "LastAxisValue": "",
    "Offset": null,
    "AxisOffset": null,
    "Reference": {
      "date": "2021-04-23T11:23:00+03:00",
      "values": [
        2671812.1077
      ]
    },
    "AxisReference": null,
    "LastUpdateTime": "2021-06-28T09:09:07.725+03:00",
    "ValueAlarmState": [],
    "SensorState": 0,
    "SensorHealth": "WDExpried",
    "SensorLink": "https://www.google.com",
    "SensorLinkName": "Google"
  }
}
```

Fields description:

- **"SensorFullId"** - fullId of sensor. Type: *string*.
- **"Properties"** - dictionary with all requested properties values.

04.12.2025

Each property is a Key-Value pair. Type of value depends on property

Key	Property description	Value type
SensorType	Type of sensor (Type of Device in case of virtual sensor)	SensorCategory or string
Customer name	Sensor's customer name	string
Unit	Unit of sensor's measurements	PhysicalUnit or string
PublicFlag	Flag shows if sensor is public	boolean
Coordinates	Map coordinates of sensor	GeoLocation
CoordinatesSystem	Map coordinates system	CoordinatesSystems
CoordinatesIn-WGS84	Map coordinates of sensor in WGS 84	GeoLocation
Color	Sensor color	string
UseColorToMap-Flag	Use sensor color to map flag	boolean
Distance	Distance	DistanceModel
Param1	Sensor's custom param 1	string
Param2	Sensor's custom param 2	string
Param3	Sensor's custom param 3	string
Param4	Sensor's custom param 4	string
Param5	Sensor's custom param 5	string
LastValue	Last RAW value of sensor	Key-Value pair: <i>string</i> : <i>double</i> []
LastAxisValue	Last AXIS value of sensor	Key-Value pair: <i>string</i> : <i>double</i> []
Offset	Last valid RAW offset	SensorAttributeEntry
AxisOffset	Last valid AXIS offset	SensorAttributeEntry
Reference	Last valid RAW reference	SensorAttributeEntry
AxisReference	Last valid AXIS reference	SensorAttributeEntry
LastUpdateTime	Time when sensor get last value	DateTime string
ValueAlarmState	Info about alarm of the sensor	Array of AlarmSensor-Model
SensorState	Sensor's measurement state	SensorState
SensorHealth	Sensor's WD health	string
SensorLink	Sensor link	string
SensorLinkName	Sensor link name	String

04.12.2025

LocH	Loc H coordinate of sensor	Double
LocY	Loc Y coordinate of sensor	Double
TunnelMeter	TunnelMeter coordinate of sensor	Double
Angle	Angle coordinate of sensor	Double
Radius	Raduis coordinate of sensor	Double

4.14. SyncRequest

Structure with sync data

JSON document sample:

```
{
  "DatabaseId": "368_0",
  "SyncType": 1,
  "DataOverwriteType": 0,
  "SyncData": [
    {
      "Id": "sensor_1",
      "Data": [1, 2, 3],
      "ExactTime": "21.12.2021 00:00:01",
      "TimeSlot": "21.12.2021 00:00:01",
      "ValueType": "double",
      "Flag": 1,
      "State": 0
    },
    {
      "Id": "sensor_1",
      "Data": [1, 2, 3],
      "ExactTime": "21.12.2021 00:00:03",
      "TimeSlot": "21.12.2021 00:00:03",
      "ValueType": "double",
      "Flag": 1,
      "State": 0
    }
  ],
  "SyncSensors": [
    {
      "Id": "sensor_2",
      "IsVirtualSensor": false,
      "SyncProperties": [
        {
          "Code": 1,
          "Value": "Test sensor_1"
        },
        {
          "Code": 9,
          "Value": 10
        },
        {
          "Code": 10,
          "Value": "Param 1"
        },
        {
          "Code": 0,
          "Value": "Prism"
        },
        {
          "Code": 2,

```

04.12.2025

```

    "Value":3
  },
  {
    "Code":4,
    "Value":{
      "Latitude":1,
      "Longitude":1,
      "Height":1
    }
  }
]
}
}

```

Fields description:

- **"DatabaseId"** - identifier of project's database. Can not be skipped. Type: *string*.
- **"SyncType"** - type of synchronization. Can not be skipped. Type: [SyncType](#)
- **"DataOverwriteType"** - defines handling algorithm for sensor's data duplicates. Can be skipped. Type: [SyncDataOverwriteType](#)
- **"SyncData"** - sensor's data for synchronization. Can be skipped in case of Sync Properties request. Type: *array of* [DatalItem](#)
- **"SyncSensors"** - sensor's properties for synchronization. Can be skipped in case of Sync Data request. Type: *array of* [SensorConfiguration](#)

4.15. DatalItem

Structure of single sensor data record

JSON document sample:

```

{
  "Id": "sensor_id",
  "Data": [1,2,3],
  "TimeSlot": "2022-01-14T06:26:30Z",
  "ExactTime": "2022-01-14T06:26:30Z",
  "Flag": 1
  "State": 0
}

```

Fields description:

- **"Id"** - id of sensor. Type: *string*.
- **"Data"** - sensor's data. Type: *array of double*.
- **"TimeSlot"** - record's timeslot. Type: *date in* [ISO 8601](#) *string format*.
- **"ExactTime"** - exactly measured time of sensor. Type: *date in* [ISO 8601](#) *string format*.
- **"Flag"** - sensor sync flag 0=Unknown / 1=Reference / 2=Offset.
- **"State"** - measured record state. Type: [MongoDataState](#).

4.16. SensorAttributeEntry

Structure for sensor's references and offsets

JSON document sample:

```

{
  "Date": "2022-01-14T06:26:30Z",

```

04.12.2025

```
"Values": [1,2,3]
}
```

Fields description:

- **"Date"** - Entry start work time. Type: date in [ISO 8601](#) string format.
- **"Values"** - vector with values. Type: array of double.

4.17. SensorPropertyModel

Structure of single sensor's property value

JSON document sample:

```
{
  "Code": 1,
  "Value": "sensor_Customer_Name"
}
```

Fields description:

- **"Code"** - sensor's property code. Type: [SensorProperties](#)
- **"Value"** - sensor's data. Type: object.

Value type depends on property code.

Code	Property description	Can be synchro- nized	Value type
0	Type of sensor (Type of Device in case of virtual sensor)	Yes	SensorCategory or string
1	Sensor's customer name	Yes	string
2	Unit of sensor's measurements	Yes	PhysicalUnit or string
3	Flag shows if sensor is public	Yes	boolean
4	Map coordinates of sensor	Yes	GeoLocation
5	Map coordinates system	Yes	Coordinates-Systems
6	Sensor color	Yes	string
7	UseColorToMapFlag	Yes	boolean
8	Distance	Yes	DistanceModel
9	Sensor's custom param 1	Yes	string
10	Sensor's custom param 2	Yes	string
11	Sensor's custom param 3	Yes	string
12	Sensor's custom param 4	Yes	string
13	Sensor's custom param 5	Yes	string
14	Last RAW value of sensor	No	

04.12.2025

15	Last AXIS value of sensor	No	
16	Last valid RAW offset	Yes	SensorAttributeEntry
17	Last valid AXIS offset	Yes	SensorAttributeEntry
18	Last valid RAW reference	Yes	SensorAttributeEntry
19	Last valid AXIS reference	Yes	SensorAttributeEntry
20	Time when sensor get last value	No	
21	Info about alarm of the sensor	No	
22	Sensor's measurement state	Yes	SensorState
23	Sensor's WD health	No	
24	Sensor link	Yes	string
25	Sensor link name	Yes	String
26	Coordinates in WGS 84	No	
27	LocX	Yes	Double
28	LocY	Yes	Double
29	TunnelMeter	Yes	Double
30	Angle	Yes	Double
31	Radius	Yes	Double
31	LocZ	Yes	Double
32	LocX	Yes	Double

4.18. SensorConfiguration

Structure of single sensor data record

JSON document sample:

```
{
  "Id": "sensor_id",
  "IsVirtualSensor": false,
  "SyncProperties": [
    {
      "Code": 1,
      "Value": "sensor_Customer_Name"
    },
    ...
  ]
}
```

Fields description:

40-G-LFC

04.12.2025

- **"Id"** - id of sensor. Type: *string*.
- **"IsVirtualSensor"** – flag shows if this sensor is virtual. Type: *boolean*.
- **"SyncProperties"** - properties of sensor to be synchronized. Type: array of [SensorPropertyModel](#)

4.19. DistanceModel

Structure of sensor's distance

JSON document sample:

```
{
  "Value": 10.45,
  "Unit": 0
}
```

Fields description:

- **"Value"** – distance value of sensor. Type: *double*.
- **"Unit"** – unit of sensor's distance. Type: [DistanceUnit](#)

4.20. AlarmSensorModel

Structure of sensor's alarm state

JSON document sample:

```
{
  "AlarmName": , "Some alarm"
  "AlarmColor": "#FFFFFF"
}
```

Fields description:

- **"AlarmName"** – name of raised alarm of sensor. Type: *string*.
- **"AlarmColor"** – color of raised alarm of sensor's distance. Type: *string*

4.21. ProjectViewInfo

Structure with project view's information

JSON document sample:

```
{
  "Id": "view_id_string",
  "Name": "Test View",
  "IsPublic": true,
  "Sensors": [
    {
      "SensorFullId": "DatabaseID.SensorID",
      "Id": "101",
      "CustomerName": "101"
    },
    {
      "SensorFullId": "DatabaseID.SensorID",
      "Id": "102",
      "CustomerName": "102"
    }
  ]
}
```

04.12.2025

Fields description:

- **"Id"** – unique ID of project view. Type: *string*.
- **"Name"** – name of project view. Type: *string*.
- **"IsPublic"** – public status of project view. Type: *boolean*.
- **"Sensors"** – information about sensors of project view. Type: array of [SensorObserve-Model](#).

4.22. ProjectViewObserveInfo

Structure with project view's information

JSON document sample:

```
{
  "Id": "view_id_string",
  "Name": "Test View"
}
```

Fields description:

- **"Id"** – unique ID of project view. Type: *string*.
- **"Name"** – name of project view. Type: *string*.

4.23. DatabaseSensors

Structure used as alternative way to specify sensor's full Ids.

JSON document sample:

```
{
  "DatabaseId": "some_database_id_string",
  "SensorsIds": ["sensor_id_1", "sensor_id_2"]
}
```

Fields description:

- **"DatabaseId"** – unique ID of required database. Type: *string*.
- **"SensorsIds"** – array of sensor's Ids, from database specified in "DatabaseId", to be processed. Type: *string*.

4.24. SyncStatesPayload

Structure with data for change states of sensors data.

JSON document sample:

```
{
  "SensorsFullIds": ["DatabaseID.SensorID", "DatabaseID.SensorID", "DatabaseID.SensorID"],
  "DatabaseSensors": [{
    "DatabaseId": "Project_database_ID_1",
    "SensorsIds": ["SensorId_1", "SensorId_2"]
  }, {
    "DatabaseId": "Project_database_ID_2",
    "SensorsIds": ["SensorId_3", "SensorId_4"]
  }],
  "StartDate": "2021-06-28T09:09:07.725+02:00",
  "EndDate": "2021-06-30T09:09:07.725+02:00",
  "TimePeriod": "00-00-00 01:10:00",
  "ValueTo": 20,
  "ValueFrom": 2.5,
  "UseData": true,
  "UseAxis": false,
  "FromStates": [0, 2, 4],
}
```

04.12.2025

```
"ToState":1,
}
```

Fields description:

- **"SensorsFullIds"** - data will be returned only for these sensors. FullId (DatabaseID.SensorID) of each sensor should be sent in this field. Can be skipped if "Database Sensors" field is set. Type: *array of string*.
- **"DatabaseSensors"** – data will be returned only for specified sensors of databases. This field is alternative way to set sensors for processing. Can be skipped if "SensorsFullIds" field is set. Type: array of [DatabaseSensors](#).
- **"TimePeriod"** - time range for data selecting. Dominates *StartDate* and *EndDate*. If needed to use *StartDate* and *EndDate* should be skipped or set to "00-00-00 00:00:00". Only available format is "yyyy-MM-dd HH:mm:ss". Every value, like years, days or minutes can be any integer value. It means that TimePeriod value "0-0-1 0-100-300" will be valid (Time range in this case will be 1 day 100 minutes 300 seconds or 1 day 1 hour 45 minutes). Can be skipped. Type: *String*
- **"StartDate"** - start date of time range for data selecting. Should be less or equal than *EndDate*. Make sense only if *TimePeriod* is skipped or zero. Can be skipped in case of *TimePeriod* is not skipped or zero otherwise can **not** be skipped. Type: *string presentation of DateTime*.
- **"EndDate"** - end date of time range for data selecting. Should be greater or equal than *StartDate*. Make sense only if *TimePeriod* is skipped or zero.. Can be skipped in case of *TimePeriod* is not skipped or zero otherwise can **not** be skipped. Type: *string presentation of DateTime*.
- **"ValueFrom"** - only sensor's values greater or equal than this value will be returned. Can be skipped. Type: *double*.
- **"ValueTo"** - only sensor's values less or equal than this value will be returned. Can be skipped. Type: *double*.
- **"UseData"** – flag means that raw records will be affected. If both "UseData" and "UseAxis" set to False – error will be thrown Can **not** be skipped. Type: *boolean*.
- **"UseAxis"** - flag means that raw records will be affected. If both "UseData" and "UseAxis" set to False – error will be thrown Can **not** be skipped. Type: *boolean*.
- **"FromStates"** – list of data states which will be replaced. Can **not** be skipped. Type: [MongoDataState](#).
- **"ToState"** – state which will be set to all selected data records. Can **not** be skipped. Type: [MongoDataState](#).

4.25. DeleteSensorsDataPayload

Structure with filters for delete sensors data.

JSON document sample:

```
{
  "SensorsFullIds": ["DatabaseID.SensorID","DatabaseID.SensorID","DatabaseID.SensorID"],
  "DatabaseSensors": [{
    "DatabaseId": "Project_database_ID_1",
    "SensorsIds": ["SensorId_1", "SensorId_2"]
  }, {
    "DatabaseId": "Project_database_ID_2",
    "SensorsIds": ["SensorId_3", "SensorId_4"]
  }],
  "StartDate": "2021-06-28T09:09:07.725+02:00",
  "EndDate": "2021-06-30T09:09:07.725+02:00",
  "ExactTimeslot": "2021-06-30T09:09:07.725+02:00",
  "ValueTo": 20,
}
```

04.12.2025

```

    "ValueFrom":2.5,
    "UseData": true,
    "UseAxis": false
    "States": [0,2,4]
  }

```

Fields description:

- **"SensorsFullIds"** - data will be returned only for these sensors. FullId (DatabaseID.SensorID) of each sensor should be sent in this field. Can be skipped if "Database Sensors" field is set. Type: *array of string*.
- **"DatabaseSensors"** – data will be returned only for specified sensors of databases. This field is alternative way to set sensors for processing. Can be skipped if "SensorsFullIds" field is set. Type: array of [DatabaseSensors](#).
- **"ExactTimeslot"** – exact time for removing record. Dominates under *StartDate* and *EndDate*. If needed to use *StartDate* and *EndDate* should be skipped. Can be skipped. Type: *string presentation of DateTime*.
- **"StartDate"** - start date of time range for data selecting. Should be less or equal than *EndDate*. Make sense only if *ExactTimeslot* is skipped. Can be skipped in case if *ExactTimeslot* is not skipped, otherwise can **not** be skipped. Type: *string presentation of DateTime*.
- **"EndDate"** - end date of time range for data selecting. Should be greater or equal than *StartDate*. Make sense only if *ExactTimeslot* is skipped. Can be skipped in case if *ExactTimeslot* is not skipped, otherwise can **not** be skipped. Type: *string presentation of DateTime*.
- **"ValueFrom"** - only sensor's values greater or equal than this value will be deleted. Can be skipped. Type: *double*.
- **"ValueTo"** - only sensor's values less or equal than this value will be deleted. Can be skipped. Type: *double*.
- **"UseData"** – flag means that raw records will be affected. If both "UseData" and "UseAxis" set to False – error will be thrown Can **not** be skipped. Type: *boolean*.
- **"UseAxis"** - flag means that axis records will be affected. If both "UseData" and "UseAxis" set to False – error will be thrown Can **not** be skipped. Type: *boolean*.
- **"States"** – list of data states which will be removed. Can **not** be skipped. Type: [Mongo-DataState](#).

4.26. BaseSensorsPayload

Structure for receiving sensors data or info.

JSON document sample:

```

{
  "SensorsFullIds": ["DatabaseID.SensorID","DatabaseID.SensorID","DatabaseID.SensorID"],
  "DatabaseSensors":[{
    "DatabaseID":"Project_database_ID_1",
    "SensorsIds":["SensorId_1","SensorId_2"]
  },{
    "DatabaseID":"Project_database_ID_2",
    "SensorsIds":["SensorId_3","SensorId_4"]
  }]
}

```

Fields description:

- **"SensorsFullIds"** - data will be returned only for these sensors. FullId (DatabaseID.SensorID) of each sensor should be sent in this field. Can be skipped if "Database Sensors" field is set. Type: *array of string*.

04.12.2025

- **"DatabaseSensors"** – data will be returned only for specified sensors of databases. This field is alternative way to set sensors for processing. Can be skipped if "SensorsFullIds" field is set. Type: array of [DatabaseSensors](#).

4.27. LabelPayload

Structure to sync logbook labels.

JSON document sample:

```
{
  "Labels": [ "Label_name_1", "Label_name_2", "Label_name_2" ]
}
```

Fields description:

- **"Labels"** - list of label names to be created if they do not exist yet. Existing labels are ignored. Type: *array of string*.

4.28. LabelResponse

Structure describes a single logbook label.

JSON document sample:

```
{
  "Id": "6863a5d00017d9f0444fb3cf",
  "Name": "Label_name_1"
}
```

Fields description:

- **"Id"** - id of label. Type: *string*.
- **"Name"** - name of label. Type: *string*.

4.29. LogbookFilter

Structure contains filters for logbook selection.

JSON document sample:

```
{
  "TimePeriod": "00-00-14 00:00:00",
  "StartDate": "2020-11-24T09:09:07.725+02:00",
  "EndDate": "2022-11-28T09:09:07.725+02:00",
  "Class": [0, 5, 9],
  "Author": ["6863a5d00017d9f0444fb3cf"],
  "Label": ["Label_name_1", "Label_name_2"],
  "Confirmed": 0,
  "Public": 0
}
```

Fields description:

- **"TimePeriod"** - time range for data selecting. Dominates *StartDate* and *EndDate*. If needed to use *StartDate* and *EndDate* should be skipped or set to "00-00-00 00:00:00". Only available format is "yyyy-MM-dd HH:mm:ss". Every value, like years, days or minutes can be any integer value. It means that TimePeriod value "0-0-1 0-100-300" will be valid (Time range in this case will be 1 day 100 minutes 300 seconds or 1 day 1 hour 45 minutes). Can be skipped. Type: *String*

04.12.2025

- **"StartDate"** - start date of time range for data selecting. Should be less or equal than *EndDate*. Make sense only if *TimePeriod* is skipped or zero. Can be skipped in case of *TimePeriod* is not skipped or zero otherwise can **not** be skipped. Type: *string presentation of DateTime*.
- **"EndDate"** - end date of time range for data selecting. Should be greater or equal than *StartDate*. Make sense only if *TimePeriod* is skipped or zero. Can be skipped in case of *TimePeriod* is not skipped or zero otherwise can **not** be skipped. Type: *string presentation of DateTime*.
- **"Class"** - list of logbook classes to be returned. If empty or skipped, entries of all classes are returned. Type: array of [LogbookType](#).
- **"Author"** - list of user IDs. If non-empty, only logbooks created by these authors are returned. Type: *array of string*.
- **"Label"** - list of label names. If non-empty, only entries which have at least one of these labels assigned are returned. Type: *array of string*.
- **"Confirmed"** - confirmation status filter. Type: [LogbookConfirmed](#).
- **"Public"** - public status filter. Type: [LogbookPublic](#).

4.30. LogbookPayload

Structure of a single logbook to sync logbook request.

JSON document sample:

```
{
  "Date": "2020-11-24T09:09:07.725+02:00",
  "Labels": ["Label_name_1", "Label_name_2"],
  "Description": "Description",
  "Comment": "Comment",
  "Public": true
}
```

Fields description:

- **"Date"** - value indicating when logbook created. Can **not** be skipped. Type: *string presentation of DateTime*.
- **"Labels"** - list of label names to be assigned to the logbook. Can be skipped. If skipped, no labels will be attached. Type: *array of string*.
- **"Description"** – description of logbook. Can **not** be skipped. Type: *string*.
- **"Comment"** – comment of logbook. Can be skipped. Type: *string*.
- **"Public"** – public flag of logbook. Can be skipped. If skipped, logbook is created as public. Type: *boolean*.

4.31. LogbookResponse

Structure of a single logbook to sync logbook request.

JSON document sample:

```
{
  "Id": "6863a5d00017d9f0444fb3cf ",
  "AuthorId": "6863a5d00017d9f0444fb3cf",
  "AuthorName": "LastName FirstName",
  "Date": "2020-11-24T09:09:07.725+02:00",
  "Class": 5,
  "Labels": [
    ["Label_name_1", "Label_name_2"]
  ],
  "Description": "Description",
}
```

04.12.2025

```

    "Comment": " Comment",
    "Public": true,
    "Confirmed": null
  }

```

Fields description:

- **"Id"** - id of logbook. Type: *string*.
- **"AuthorId"** - list of user ID. Type: *string*.
- **"AuthorName"** – username (lastname firstname). Type: *string*.
- **"Date"** - value indicating when logbook created. Type: *string presentation of DateTime*.
- **"Class"** - class of logbook. Type: [LogbookType](#).
- **"Labels"** - list of label names to be assigned to the logbook. Type: *array of string*.
- **"Description"** - description of logbook. Type: *string*.
- **"Comment"** - comment of logbook. Type: *string*.
- **"Public"** - public flag of logbook. Type: *boolean*.
- **"Confirmed"** - confirmation status flag for confirmable [LogbookType](#) (FA1-FA4, FA5, FA6, LoRa). Type:
 - *true* - logbook is confirmed.
 - *false* - logbook is waiting for confirmation.
 - *null* - for non-confirmable logbook types.

4.32. InspectionDatabaseModel

Structure of a single inspection database with all inspection objects inside.

JSON document sample:

```

{
  "DatabaseId": "Inspection_DB_ID",
  "DatabaseName": "Inspection_1",
  "InspectionObjects": [
    {
      "Uid": "123",
      "Name": "Inspection object name"
    }
  ]
}

```

Fields description:

- **"DatabaseId"** - id of inspection database. Type: *string*.
- **"DatabaseName"** - name of inspection database. Type: *string*.
- **"InspectionObjects"** - objects which are created in the inspection database. Type: array of *InspectionObjectShortInfo*.

04.12.2025

4.33. InspectionObjectShortInfo

Structure for short description of inspection object.

JSON document sample:

```
{
  "Uid": "123",
  "Name": "Inspection object name"
}
```

Fields description:

- **"Uid"** - id of inspection object. Type: *string*.
- **"Name"** - name of inspection object. Type: *string*.

4.34. InspectionObjectInfo

Structure for description of inspection object.

JSON document sample:

```
{
  "LatestAssessmentRating": 3,
  "LatestAssessmentDate": "02.12.2025",
  "Name": "Test inspection with files",
  "Uid": "123",
  "Color": "#ffff00",
  "Points": []
}
```

Fields description:

- **"Uid"** - id of inspection object. Type: *string*.
- **"Name"** - name of inspection object. Type: *string*.
- **"LatestAssessmentRating"** - last assessment value of inspection object. Type: *string*.
- **"LatestAssessmentDate"** - last assessment value time of inspection object. Type: *string representation of DateTime*.
- **"Color"** - color of inspection object. Type: *string*.
- **"Points"** - map coordinates of vertexes of inspection object. Type: array of [GeoLocation](#).

Versions changes

Version 1.5.1 (13.11.2023)

- In payload for **Get sensors data request** field *"Period"* was removed. Instead of it string field *"TimePeriod"* was added.
- In payload for **Get sensors data request** and **Get sensors properties request** was added new alternative way to specify sensors to process – field *"DatabaseSensors"*.
- *"ExactTime"* of each data item of **Sync sensors data request** can now be skipped. In this case this field will be set with value of *"TimeSlot"* field.
- In each sensor data item of **Sync sensors data request** added field *"State"* for specifying state of each data record.
- In payload for **Get sensors data request** field *"OnlyOk"* was removed. Instead of it field *"States"* was added for specifying states of returned data records.
- New request added – **Change sensors data state request**.
- Added new field *"HumanReadable"* for **Get sensors properties request**. When it set to true sensor type and sensor unit will be returned as human readable strings.
- Sensor type and sensor unit can now be synchronized with string values.
- Local coordinates added as new sensors parameters for receiving and synchronization.
- Added new request **Delete sensors request** for removing sensors data records from database.

Version 1.5.2 (26.01.2024)

- New Physical units added.

Version 1.5.3 (24.09.2025)

- New Physical units and Coordinate Systems added.

Version 1.5.4 (10.11.2025)

- Added new properties for **Get sensors data request**: *"RequestedValueTime"* to specify what type of time needed in response (Timeslot or Exact) and *"RequestedValueTimeFormat"* to specify time format for time in each data entry.

Version 1.5.5 (24.11.2025)

- Added new **get logbook labels** request.
- Added **sync logbook labels** request.
- Added **get logbook** request.
- Added **sync logbook** request.
- Added new enums: LogbookType, LogbookConfirmed, LogbookPublic.
- Added new data structures: LabelPayload, LabelResponse, LogbookPayload, LogbookResponse.
- New coordinate systems added.

04.12.2025

Version 1.5.6 (01.12.2025)

- **PhysicalUnit.** Fixed duplicate/missing codes – dmPerMonth, mPerMonth; Meter-AboveSeaLevel.
- **AuthorizationCode.** Added TooBigResponse, LicensingFailed, TimeoutException.
- **Sync data.** Clarified **DatalItem** – added explicit "**State**" field; "**Flag**" semantics.
- **SensorCategory.** Added Height, Modem. Fixed typos (Piezometer, Settlement).
- **InterpolationAlgorithmMode.** Added NormalMedian, NormalMedianBackwards.
- **CoordinatesSystems.** Added ETRS_89 and EPSG_25832.
- **Payloads.** Added ExactTimeslot to SensorsDataFilter. Fixed JSON samples in 2.14/2.15/4.26.
- **Sync data.** For Prism, Theodolite, GPS, Position sensors with an axis assignment, the server recalculates AXIS values automatically after RAW synchronization. Not applied to AGMS synchronization.
- **Sync data.** If none of the sensors from "SyncData" exists in the database, the request fails with Success=false and AuthorizationCode=6 (ImpossibleOperation) with message "Sensors are missing from the project. First you need to create them".

Version 1.5.7 (04.12.2025)

- Added new **get inspection objects** request.
- Added new **get inspection object info** request.
- Added new structures: **InspectionDatabaseModel**, **InspectionObjectInfo**, **InspectionObjectShortInfo**.